Ralph:
A Dynamic Language
with Efficient Application Delivery

July 25, 1991

Author: Andrew LM Shalit
Apple Computer, Inc
238 Main Street
Cambridge, MA 02142
alms@cambridge.apple.com

Language Design: Glenn S. Burke, Robert Cassels, Jerome Coonen, James Grandy, Jed Harris, Alice K. Hartley, John Hotchkiss, Jeremy A. Jones, David A. Moon, Ike Nassi, Jeffrey Piazza, Andrew LM Shalit, David Singer, David C. Smith, Walter R. Smith, Bill St. Clair, Oliver Steele, Joshua Susser, Larry Tesler, Bob Welland, Gail Zacharias.

This work was performed at the Cambridge lab of Apple Computer's Advanced Technology Group, in consultation with other groups within Apple.

Introduction

Ralph is a new programming language which draws from previous work done in Scheme, Common Lisp, and CLOS, as well as Smalltalk and other object-oriented languages.

The Ralph language design was undertaken to make dynamic object-oriented programming practical for commercial application development. The dynamic object-oriented languages currently available—Common Lisp and Smalltalk—have memory requirements which prohibit their use for many tasks. The object-oriented language most widely used for commercial development—C++—is missing very important dynamic capabilities, such as dynamic linking, compacting garbage collection, and any form of introspection.

The overarching goal of Ralph is to be a practical language which will appeal to a large audience of application programmers and possibly system programmers. With this goal in mind, the following subgoals were adopted.

Consistency
The language must be built around a consistent model. Higher level features should be explained and implemented in terms of core capabilities. Stylistic conventions should be used to make the language easier to learn.

Objects
The language must be thoroughly object-oriented. The object system should not be an isolated part of the language, but should be apparent throughout the design of the language as a whole.

Efficiency
The language must be capable of delivering applications that are efficient both in space and speed.

Dynamism
The language should be fully dynamic. It should support incremental linking, dynamic memory allocation and garbage collection, runtime type checking, runtime error recovery, and runtime changes to classes with updating of instances. It should provide introspective features to support the highest quality programming and debugging tools. Compiled code should support tight integration with runtime scripting systems and component architectures.

Completeness and Parsimony
The language must be useable "out of the box". The language model and core libraries must be expressive enough that programmers do not feel compelled to extend the language before they can use it. At the same time, the language should not be so large as to present a confusing array of overlapping features to the potential user.

Large Systems
The language must include namespace management features, to support group programming and the development of large systems.

With these goals in mind, there arose two "non-goals":

Compatibility
Compatibility with previous dialects of Lisp was not a goal.

Academic Direction
We did not include features that are of academic interest, but which lack broad practical applications and compromise efficiency. For example, two features of Scheme which Ralph does not include are upward continuations and imprecise integers.

Language Overview

Ralph attempts to find a middle ground between the plethora of features found in Common Lisp with CLOS and the spartan libraries of Scheme.

Control structures and stylistic conventions were taken from Scheme [5]. Ralph extends the core semantics of Scheme with objects, macros, keyword arguments to functions, and the ability to return multiple values.

The object system is based primarily on CLOS [14]. It is built around classes and generic functions, and includes multiple inheritance, multi-methods, and methods on individual objects.

A module system provides namespace management by controlling access to variables.

Ralph supports a rich numeric library with user-defined numeric classes, drawing on work done in Common Lisp, Scheme, and EuLisp [11]. A class-based exception handling mechanism draws on work done by Kent Pittman [12]. A moderate sized collection library draws on the Smalltalk collection classes, Common Lisp sequence operations, and Richard Waters' Series operations [16] [17]. There is a small assortment of other classes and operations.

Ralph uses the parenthesized notation traditional to Lisp. We are considering support for an Algol-like syntax as well.

Because compatibility with an existing language was not a requirement, Ralph required fewer compromises. Features adopted from other languages were redesigned to complement each other, reflect the core language model, and remove redundancies.

Anonymous Meta Objects

To achieve its efficiency goals, Ralph provides a secure encapsulation system and strict control of the dynamic features of the language.

Encapsulation is provided through a threefold approach:

> Meta objects such as classes, generic functions, methods and modules are anonymous. Just like other Ralph objects, they are stored in variables or other data structures, and can only be accessed through the standard data access primitives, such as variable reference. Meta objects do not have "global names" through which they can be looked up.
>
> Access to variables can be strictly controlled by the module system. A module maps variable names to variable locations. Unlike Common Lisp, Ralph uses a single namespace for all classes of objects. It does not have one namespace for variables, another for classes, another for functions, another for modules, etc. The use of a single namespace simplifies data flow analysis.
>
> A set of "sealing" operations can be used to prohibit runtime redefinition and introspection of objects.

Strictly controlling access to meta objects lets Ralph deduce that certain dynamic operations will never be performed on many objects. In addition, explicit sealing commands can provide this information for other objects. The combination allows a number of optimizations that would not otherwise be possible, such as precomputing method dispatch and compressing class representations.

Anonymous objects and a single namespace also help simplify namespace management. The problem of preventing name conflicts between classes, functions, modules, etc is reduced to the problem of avoiding variable name conflicts, which in turn is handled by the module system.

First class anonymous meta objects also keep the evaluation and scoping semantics simple. Operations on meta objects do not require special syntax. They are often handled by standard function calls.

The Object System

Classes and Generic Functions

Ralph classes support multiple inheritance. A subclass can inherit behavior as well as structure from multiple superclasses. Classes can be redefined at runtime, and existing instances will be updated to reflect the new class definition. Redefinition can include changing the superclasses of the class, as well as changing the local storage allocated for instances of the class.

Like other recent object-oriented dialects of Lisp [10] [11] [14], Ralph implements polymorphism through generic functions, rather than through message-passing. A generic function contains a number of methods. When a generic function is called, a method is selected based on the types of the arguments.

The applicability of a method is determined by the specializers of the method. Specializers associate types (usually classes) with parameters of the method. A method is applicable if each specialized parameter matches the type of the corresponding argument. For example, the following expression defines a method which is applicable to instances of `<number>`.

```
(define-method double ((thing <number>))
  (+ thing thing))
```

Ralph methods can be specialized on more than one parameter, and a generic function can contain methods which specialize different parameters. This capability is called "multi-methods". The following example shows how multi-methods might be used to implement a two argument addition operation.

```
(define-method two-arg-+ ((n1 <small-integer>)
                           (n2 <small-integer>))
  (primitive-small-integer+ n1 n2))

(define-method two-arg-+ ((n1 <float>) (n2 <float>))
  (primitive-float+ n1 n2))

(define-method two-arg-+ ((n1 <float>) (n2 <small-integer>))
  (primitive-float+ n1 (as <float> n2)))

(define-method two-arg-+ ((n1 <small-integer>) (n2 <float>))
  (primitive-float+ (as <float> n1) n2))
```

Anonymous Classes

Performing standard evaluation on class expressions introduces two minor problems. The first is that there is an increased likelihood of name conflicts. For example, Ralph cannot have a class named `list` as well as a function named `list`. There is only one `list` variable, and it can only store one value. Ralph circumvents this problem through the use of a naming convention. In Ralph, variables which hold classes begin and end with angle brackets. The class is therefore stored in the variable `<list>`, and the function is stored in the variable `list`. After a period of use, we found that this convention also improved code readability.

A second difficulty involves order of evaluation. Because class expressions are evaluated, programs cannot have forward references to classes. CLOS does not have this problem, because classes are normally referenced by quoted symbols, where the symbol is the name of the class. We solved this problem by introducing delayed evaluation into the defining forms which reference classes. The class expressions are closed over the definition environment, but they are not evaluated until the class is needed at runtime.

Slots

Slots are used to store the information associated with an instance. They correspond to the instance variables of other object-oriented languages.

In Ralph, all slots are accessed through function calls. Slot values are retrieved through a function of one argument (the instance) and they are set through a function of two arguments (the instance and the new value). This approach to slots is similar to that of the language Self [15], which uses the same syntax for accessing instance variables as for sending other messages, and CLOS, which encourages but does not enforce a functional approach.

A functional view of slots clarifies the fact that slots are part of the interface to an instance, and that the implementation behind the interface should remain hidden. All slots provide the same interface: a getter generic function and setter generic function. But some slots are implemented with storage in the instance, while other slots are computed. Clients of an instance do not need to know which technique is used to implement the slot. This abstraction barrier lets different classes choose different implementations for the same slot. The implementation of a slot can be changed over time, and clients of the class (including most of the code which implements the class) does not need to change, because the interface remains the same.

When a program declares a slot, it specifies whether storage for the slot should be allocated in instances. If not, the program must explicitly define methods to access and retrieve the slot value. In cases where storage is not requested, the slot definition serves documentation purposes. This information is useful for inspectors, browsers, and other programming tools.

If a slot is declared to have a type, the method added to the setter generic function is specialized on the second argument, the new value. This restricts the values that can be stored in the slot, and gives the compiler reliable information about the types of values that may be returned from slot access. The following class definition includes a slot `list-storage`, which is typed to always contain a `<list>`.

```
(define-class <boxed-list> (<sequence>)
  (list-storage init-value: '()
                type: <list>))
```

The slot setter for the `list-storage` slot is defined as if with the expression

```
(define-method (setter list-storage) ((b <boxed-list>)
                                      (new-value <list>))
  ... ;code to store in the instance
  )
```

The getter and setter generic functions are stored in distinct variables, so it is possible to export the getter and setter independently. For example, the implementor of the `<boxed-list>` class might choose to export and document the getter for `list-storage`, while leaving the setter hidden.

Singletons

In object-oriented programming, it is often useful to specialize the behavior of individual objects, in addition to specializing whole classes of objects. This capability is available in Self, Object Lisp [6] [13], and to a limited extent in CLOS.

Ralph provides a simple mechanism which allows the behavior of individual objects to be specified. Rather than taking the syntactic approach of CLOS, Ralph uses a functional approach, thus simplifying the integration of individual methods with the rest of the language.

The expression `(singleton` *object*`)` returns a singleton for *object*. Most defining operations allow a singleton to be used in place of a class. When a singleton is used, the operation is applied to the individual object (the singleton's object), rather than to all the instances of a class. With this technique, methods can be specialized on individual objects and slots can be added to individual objects.

Modules

Ralph modules are used for creating separate namespaces for programs and program elements. The module system operates at the level of variables. Unlike module systems in some other languages, Ralph modules do not operate on higher semantic units, such as classes, functions, and operation signatures.

A module is essentially a mapping from variable names (usually symbols) to variable cells. The same variable name may map to different variable cells in different modules. This mapping is performed at compilation time; the module is part of the compile-time environment.

Because all Ralph objects, including classes and functions, are anonymous, controlling access to variables is usually a sufficient technique for controlling access to all program elements. To make a class or function visible outside a module, you export the variable containing it. If the variable is not exported, the class or function will not be readily available to clients outside the module. Sealing operations (described below) can be used to prevent access through the introspective features of the language.

Ralph modules are themselves anonymous objects. This allows modules to be hidden, and prevents name conflicts when large systems are built by different programmers. A program can only access a module if it has a way to reference it.

The operations available on modules include adding and removing variables, locking variables (making them read-only), exporting variables, and using other modules. When one module uses another, it gains access to all the exported variables of the other module. The `use` operation can specify renamings of variables. This allows the receiving module to name the variables with symbols that are different from the symbols used to name the

variables in the supplying module. This renaming capability allows the resolution of name conflicts that would otherwise require code to be rewritten and recompiled.

For example, the following expression imports all the exported variables from `graphics-module` into `my-module`. The two modules will use the same names to refer to all the variables, except that the variable called `frame` in `graphics-module` will be called `g-frame` in `my-module`, and the variable called `draw` in `graphics-module` will be called `g-draw` in `my-module`.

```
(use-module my-module graphics-module
            rename: '((g-frame . frame)
                      (g-draw . draw)))
```

The `use` operation can also limit which variables are imported, and it can specify a uniform renaming to be performed on all the variables imported. For example, the following expression imports all the exported variables from `graphics-module` into `my-module`, but it prepends all their names with `g-`.

```
(use-module my-module graphics-module
            prefix: "g-")
```

A module can export variables which were created in it, or which were imported from another module. Thus, it is possible to create interface modules whose only purpose is to selectively export variables which were imported from other modules. This allows an implementation to be made available with more than one interface. For example, a class library can present one interface to subclassers and another to other clients.

Object-oriented languages often conflate module abstraction with class abstraction. In such a system, class boundaries provide the only interface boundaries; code is either internal to a class, or external to it. This pattern of abstraction can be accomplished with the Ralph module system, at the discretion of the programmer. However, it is often the case that interfaces cross class boundaries, that a component will use facilities of several classes, or that only portions of a class should be made available to outside users or subclassers. All of these factors call for a module system that is distinct from the class inheritance mechanism.

Sealing

Optimization of object-oriented languages is difficult, because a given operation may dispatch to any of a number of methods, depending on the classes of the arguments. This is made even more difficult in dynamic object oriented languages, where methods can be added or removed at run time, and classes may be redefined.

Sealing is a mechanism for limiting the dynamic behavior of Ralph programs. It supports optimizations and security through the enforcement of interface boundaries.

The sealing operations are specialized for different classes of objects. In general, sealing limits the dynamic capabilities of an object. It usually makes the object read-only, and blocks the use of introspective operations[1]. Sealing allows objects to be given a more

[1]The ability to make an object read-only is also available as a separate operation for some classes of objects.

compact representation, it guarantees that the representation of an object will not change, and it prevents other programs from crossing interface boundaries.

Self performs optimizations with aggressive on-the-fly recompilation and caching techniques [2] [3]. These techniques were unsuitable for our purposes, because they require large memory resources. The space consumption of dynamic languages has always been a factor limiting their use, something we want to overcome with Ralph. A compromise was found in the fact that dynamic operations are most widely used during development, rather than at application runtime. By having well-defined sealing semantics, a programmer can specify which objects require dynamic capabilities and which objects do not. The cost is only incurred for those cases where the feature is used.

Sealing Generic Functions

Sealing generic functions prevents methods from being added, removed, or changed for a specified branch of the type graph. So, for example, you could seal the generic function `size` for the signature `(<list>)`. This would mean that no methods specializing on `<list>` or any of its subclasses can be changed in `size`[2]. Sealing gives the compiler knowledge of the behavior of a generic function when applied to certain classes of arguments. It also lets the implementor of a generic function ensure that its behavior will not change.

Methods on a sealed branch of a generic function are not visible to the introspective features of Ralph. If there are several methods on the branch, they may appear as a single method. This allows the compiler to save space and possibly speed by compiling all of these methods into a single piece of code.

Sealing Classes

Sealing a class makes it read only. This prevents the addition or removal of slots, and prevents the inheritance path of the class from being changed. In addition, introspective operations on the class are blocked. The subclasses and superclasses of the class can no longer be retrieved, and the list of the slots in the class is hidden.

This hiding accomplishes two things: it allows the representation of the class to be compressed, and it guarantees that the abstraction boundaries created by the module system will not be circumvented. Without sealing, implementation-specific superclasses, subclasses and slot accessors could be retrieved, even if variables containing them were not exported. Programs could use the introspective features of Ralph to retrieve these hidden program elements, thus compromising the integrity of a component.

In addition to sealing, it is possible to prevent a class from being subclassed. In the presence of multiple inheritance, this is necessary to guarantee that the slots in instances of the class will always be at the same location in the instance.

These sealing techniques provide an explanation for the semantics of built-in classes which have a special representation, such as small integers and vectors. Smalltalk has several such classes, but they are not documented and cannot be explained in terms of the language. Common Lisp with CLOS has such classes, but they are merely described as "built-in" classes. In Ralph, the restrictions on these classes can be explained in terms of

---

[2]The limitation also applies to singleton methods, where the singleton objects are general instances of `<list>`.

the facilities available in the language. Ralph programmers can place the same restrictions on classes which they create.

Sealing Modules

When a module is sealed it is made read-only, and the list of modules which it uses is hidden. By hiding the used modules, it is possible to expose an interface module and make the implementation module completely inaccessible.

Variables which have not been exported from the module are also hidden; they are no longer accessible to introspective tools, nor are they accessible to the compiler.

Making a module read-only means that no variables can be added or removed, no new modules can be used, and no modules currently used can be unused. This guarantees that the structure of the module will not change. Making a module read-only does not mean that the values of variables in the module cannot have their values changed. The variables can still be assigned, but variables cannot be added or removed from the module.

Sealing modules allows a number of optimizations. At the simplest level, symbols which name internal variables can be removed, thus saving space. In addition, further analysis can be performed on the code of the module as a whole. A sealed module can undergo block compilation and global data flow analysis. This can yield information that certain classes or functions will never escape from the module, and so can themselves be sealed.

Collections

The abstract class `<collection>`, and its subclass `<sequence>`, are defined to support simple access and iteration protocols. Concrete subclasses must implement these protocols. Some classes, such as lists and vectors, are implemented with direct storage. Other collection classes, however, do not require storage for each of their elements. They implement the protocols through other computations. For example, a `<range>` is a computed sequence which maps from successive indices to successive elements of a (possibly infinite) numeric series. A `<displaced-sequence>` accesses elements of a portion of another sequence.

Ralph programmers can easily define new subclasses of `<collection>` which perform operations to compute their contents as needed. This is a powerful tool for delayed evaluation and delegation. By supporting the small set of protocols, the full collection operation suite is available for these objects. The definition of the protocols allows many of these operations to be optimized, eliding the need to allocate storage for collections used as intermediate values in nested collection operations.

Exception Handling

A long-standing problem of software engineering is to develop an organized way to deal with exceptions, situations that must be handled gracefully but that are not conceptually part of the normal operation of the program [12].

The Ralph condition system organizes exceptional situations through classes. Exceptions are signalled by creating a condition object of the appropriate class, and passing it to a signalling function. Programs can install handlers to respond to classes of conditions. A handler responds to a condition by returning control to the signaller, passing control to

another point in the program with a non-local exit, or by deferring and giving other handlers a chance to handle the condition. Possible recovery strategies can be registered as handlers for subclasses of `<restart>`.

The exception handling mechanism has the advantage that it separates code for normal execution from code for exceptional situations. It also guarantees that all serious exceptions will be noticed, solving the problem of error codes being ignored for errors which the programmer assumes are impossible.

Future Work

Currently, all the classes and operations in Ralph are defined in a single module. In the future, we plan to split the language into several modules. This will further expose the cost of dynamic operations. For example, the use of some introspective operations requires that additional information be retained about an object. If all introspective operations were in a "programming environment support" module, then programmers could choose to not use this module in their final applications.

There is substantial work to be done in deciding which built-in classes and generic functions are sealed, and which ones can be specialized by Ralph programmers.

We are considering support for a full meta-object system in Ralph. Only the most rudimentary work has been done on this.

The macro system for Ralph has not yet been defined. Our work will be based on the designs currently being discussed by the Scheme community [1] [4] [9].

Ralph types are currently restricted to classes and singletons. It would be useful to have a more expressive type system. For example, it would be useful to declare that a value is a number between 1 and 100, or that a value is a boolean or a number. This would allow further optimizations.

Summary

Ralph is a full featured dynamic object oriented language built around a relatively small, consistent set of principles. By controlling access to objects and dynamic capabilities, Ralph allows a number of speed and space optimizations which are not possible in other dynamic object oriented languages. These optimizations increase the likelihood of making dynamic object oriented programming practical for commercial software development.

References

[1] Alan Bawden and Jonathan Rees.
Syntactic Closures. In *Proceedings of the 1988 ACM Conference on Lisp and Functional Programming*.

[2] Craig Chambers and David Ungar.
Customization: Optimizing Compiler Technology for Self, a Dynamically-Typed Object-Oriented Programming Language. In *Proceedings of the SIGPLAN '89 Conference on Programming Language Design and Implementation*.

[3] Craig Chambers, David Ungar, Elgin Lee.

An Efficient Implementation of Self, a Dynamically-Typed Object Oriented Language Based on Prototypes. In *OOPSLA '89 Conference Proceedings*.

[4] William Clinger and Jonathan Rees.
Macros That Work. In *Proceedings of the 1991 ACM Conference on Principles of Programming*.

[5] William Clinger and Jonathan Rees, editors.
Revised$^{3.99}$ report on the algorithmic language Scheme. University of Oregon technical report CIS TR-90-02.

[6] Gary Drescher.
Object-Oriented Logo. In *Artificial Intelligence and Education: volume I, Learning Environments and Tutoring Systems*. Ablex, Norwood, NJ (1987).

[7] Margaret A. Ellis and Bjarne Stroustrup.
The Annotated C++ Reference Manual. Addison-Wesley, Reading MA (1990).

[8] Adele Goldberg and David Robson.
Smalltalk-80: The Language and Its Implementation. Addison Wesley, Reading MA (1983).

[9] Eugene Kohlbecker, Daniel P. Friedman, Mathias Felleisen, and Bruce Duba.
Hygienic Macro Expansion. *In Proceedings of the 1986 ACM Conference on Lisp and Functional Programming*.

[10] Kevin Lang and Barak Pearlmutter.
Oaklisp, an Object-Oriented Dialect of Scheme. Lisp and Symbolic Computation, volume 1, number 1 (June, 1988). Kluwer Academic Publishers, Boston.

[11] Julian Padget and Greg Nuyens, editors.
The EuLisp Definition. Unpublished.

[12] Kent M. Pitman.
Exceptional Situations In Lisp. In *Proceedings of the First European Conference on the Practical Application of Lisp* (1990).

[13] Andrew LM Shalit
Macintosh Allegro Common Lisp 1.3 Reference. Apple Computer, Cupertino, CA (1989).

[14] Guy L. Steele Jr.
Common Lisp the Language, Second Edition. Digital Press, Burlington MA (1990).

[15] David Ungar and Randall B. Smith.
Self: The Power of Simplicity. In *OOPSLA '87 Conference Proceedings*.

[16] Richard C. Waters.
Optimizations of Series Expressions, Part I: User's Manual for the Series Macro Package. AI Memo 1082. MIT Artificial Intelligence Laboratory, Cambridge, MA (1989).

[17] Richard C. Waters.
Optimizations of Series Expressions, Part II: Overview of the Theory and Implementation. AI Memo 1083. MIT Artificial Intelligence Laboratory, Cambridge, MA (1989).